

A data pipeline with Apache Airflow and Dask

Cite as: Martin Paul Eve, "A data pipeline with Apache Airflow and Dask", *eve.gd: Martin Paul Eve*, March 09, 2023, <https://doi.org/10.59351/5jkka-69w74>.



In my [new role at Crossref](#) I work on a series of data pipelines for research and development projects. These are resource-intensive data processing tasks that need to be executed periodically on a schedule, with good observability, but also with parallel processing capacity.

Amazon's Managed Workflows for Apache Airflow (MWAA) seems like an ideal solution for this. It allows you to write DAGs (directed acyclic graphs) that define tasks that are then executed, with parallelization happening when possible.

However, there are some serious limitations of Airflow:

- You can't dynamically parallelize. That is, if you split your data into chunks, you can't tell Airflow to create a dynamic number of parallel tasks based on varying data size.
- Deployment is a total pain. You have to package your python modules into pip and then import them into Airflow. It makes for a miserable developer experience.

Another technology, Dask, does the first of these much better. The task graph in Dask is dynamic and can also be split across multiple machines, giving enhanced processing power when needed. Alternatively, it can run locally, so you can develop on a single machine, and then scale the process to a distributed environment. However, Dask does not handle the scheduling of Airflow. I also wanted a system that would allow us to deploy instantly to Airflow from a git repository, rather than needing to package everything.

To merge these two and get a solution that works for us, I put together two projects:

- [An Airflow launcher](#)
- [A Dask-provider library](#)

Essentially, the Airflow script bootstraps git onto AWS, pulls your git repository down, syncs requirements across Airflow and the AWS Fargate cluster, initializes said Fargate cluster of the number of machines that you want with however much RAM and CPU-age you want, and then finally calls your specified code entry point, passing the Dask client reference so that you can multiprocessing. Because it's a context manager, it handles automatic shutdown/scale-to-zero of the cluster. You can also pass `local=True` to the DaskRunner to operate in local testing mode where it doesn't spin up the AWS resources, but runs in separate processes locally. So the idea is that this is easy to develop with locally, and then easy to deploy by changing one flag on the constructor.

I've also written nearly 100% test coverage on the DistRunner library, so I have at least some confidence in its operation!

In terms of ongoing maintenance, the biggest problem is the git bootstrapping. AWS MWAA doesn't have git binaries on the base image (Linux-4.14.301-224.520.amzn2.x86_64-x86_64-with-glibc2.26). This uses a precompiled binary. The way I did it was to start an EC2 box with matching kernel version of Amazon Linux, install git and extract the binary and the git-core folder (`/usr/libexec/git-core`), package the latter in a zip, and publishing on our GitLab repo. If they update the glibc version on the MWAA box, we will need to extract new git binaries for this to work.

I also had, in the DaskRunner, to patch a set of bugs in Dask. The most annoying of these is a set of [errors spewing to the console after shutdown of the Dask cluster](#). This appears to be related to weakref finalizers that don't correctly check the state of the distributed computing client. I intend to write to the Dask upstream community about this and to try to fix there, but in the meantime, this library monkey patches a set of fixes against Dask AWS 2022.10.0. Sure, that's not a good way to do it in the long term, but it works for now!

In any case, I intend to start deploying some of our recurrent data tasks to this framework to evaluate its stability and ongoing suitability for these pipelines. I will also, then, be able to benchmark our operations against the local/single machine versions.